How I build with AI: guardrails first

Running AI as an engineering team, with safety, privacy and the limits on what a system may do settled before the first line of code is written.

Rory McClelland · mcclelland.info

October 2026

4 days
from the first version of Job Scout to version 0.12

14
releases, each one ending in working software

244
automated tests: 131 for the code, 113 for the checking tools

0
applications, replies or messages sent on my behalf

I lead product. When I build software, Claude is the engineering team, and I run it the way I would run a human one: a ranked backlog, short sprints, a definition of ready, tests before code, and a close that leaves nothing stranded. The speed is real, but it is not the interesting part. What makes the approach worth writing up is the guardrails: what a system must never do, which part of it may see which data, how much authority an AI is given and how it earns more, and how every claim about the system is proved rather than asserted.

This note sets out that way of working, using two things I have built in this way: **Job Scout**, an automated job search that runs every morning on my own mailbox and career record (the subject of the companion case study), and a **task oversight app** that brings tasks scattered across several tools into one view. In both cases I designed the system and made every decision; Claude wrote the code, which I read and questioned as it went.

01

Three verdicts: yes; yes, but; or blocked.

Where the instinct comes from

My habits around guardrails were formed well before AI coding tools existed. As Chrome's first privacy product manager at Google, I led the product side of the privacy review that every Chrome launch had to clear. It returned one of three verdicts: yes; yes, but, with changes required; or blocked. Engineers sat in the review, which made it a technical gate rather than a policy sign-off. Later, at Bumble, I shipped proactive toxicity detection in chat, where the hard part was not the model but measuring its mistakes: false positives through in-chat disputes and human review, and false negatives through user reports.

Two lessons from that work shape everything that follows.

> **"I want privacy" cannot be solved until you ask: private from whom, and whom is the person willing to trust to get it?**

The second lesson is that privacy and safety are sometimes perfectly aligned and sometimes flatly opposed. Neither can be maximised blindly, so the trade-off has to be made deliberately, by a named person, and written down where the next person can find it.

02

Start with the job, and with the never list

Every build begins with the job I am trying to get done, not with features. Before a single story is written, three things are agreed in one sitting: how success will be measured, the

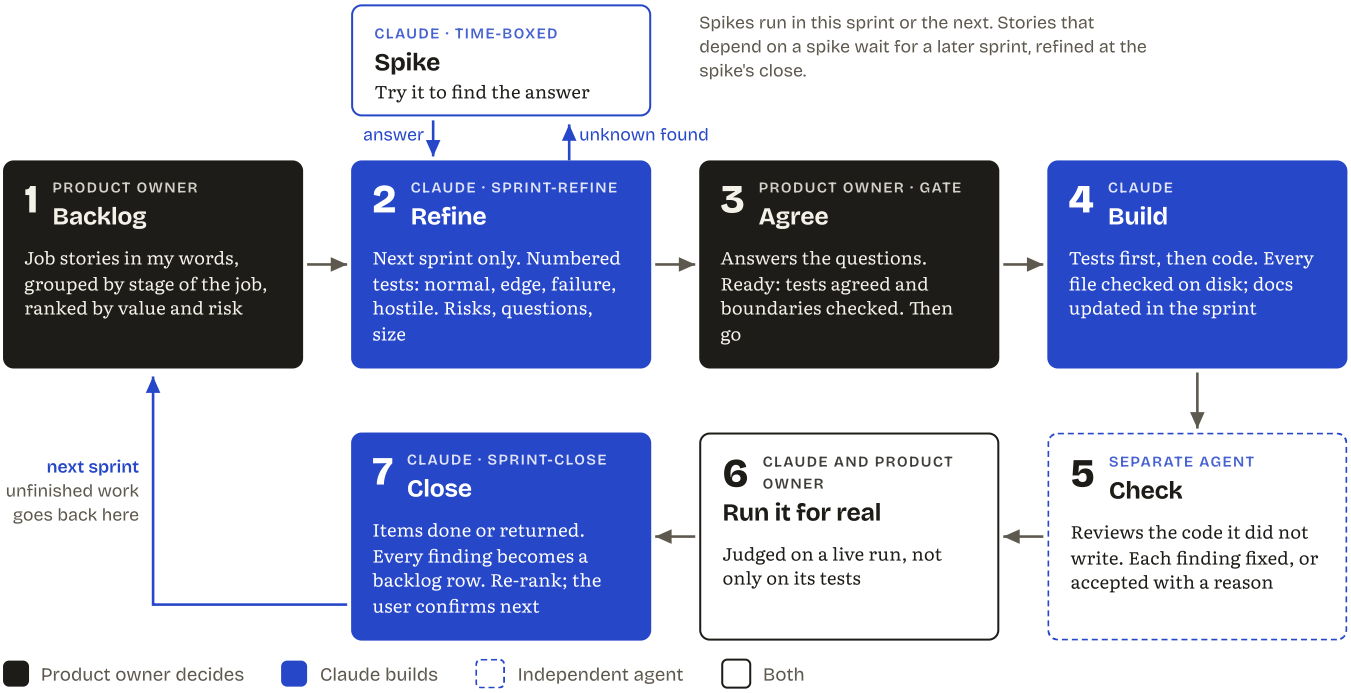
constraints, and what the system must never do. For Job Scout the job was *"run a senior job search in 30 minutes a morning"*, the success measures were relevant roles found per day and applications actually made, and the never list was short and absolute:

NEVER	NEVER	NEVER	NEVER
apply for a role	reply to or follow up any message	edit my LinkedIn profile or a sent document	share the board or change the rules

Stories are then written as job stories, in my own words: the situation, what I need done and why. *"Find the emails about my applications, and only those"* is a job; *"build an IMAP filter"* is not. Epics follow the stages of the job in order, and the first sprint is a thin slice through every stage, so that something genuinely usable exists by the next morning. Later sprints deepen it.

03 The cycle

Each sprint follows the same loop. I own the backlog and the two gates: nothing is built until I have agreed the refined stories, and nothing new starts until I have confirmed the next sprint. Refining and closing a sprint are written as reusable Claude skills, so every project runs them the same way.



Refine to a definition of ready. Each story gets numbered test cases (normal, edge, failure and hostile), its risks, its open questions and a size. A story is ready only when the tests are agreed and the boundaries have been checked against it.

Unknowns are spiked, not guessed. A time-boxed spike answers the question by trying it. A story that depends on a spike never shares the spike's sprint; it is refined at the spike's close, on what was actually learned.

Judged on a live run. Each sprint is closed against real use, not only its tests, because real use finds what tests cannot (more on that below).

Nothing stranded. At the close, unfinished work goes back to the backlog, every review finding becomes a backlog row the same day, and the whole backlog is re-ranked before the next sprint is confirmed.

04 Seven guardrail principles

These are the rules I apply to every build, and especially to anything that uses AI. None of them is new; most come straight from security and product practice. What AI changes is how easy they are to skip, because a working demo arrives so quickly.

1 · ENFORCE THE NEVER LIST

The never list lives in code, not only in instructions

An instruction can be ignored or overridden; a missing capability cannot. Limits are written into every agent's instructions and then made true by design.

In practice No part of Job Scout has a tool that can send, apply or reply. The CV builder fails the build if a claim I have barred appears.

2 · MAP THE DATA FIRST

Decide where each kind of data may go before deciding what to build

The first architectural question is not "what can it do?" but "which part may see what?". Each kind of data gets a boundary, and the design follows from those boundaries.

In practice My mailbox is read only by code on the local PC. My mail password is reachable by nothing, AI or otherwise.

3 · AGENTS ONLY WHERE NEEDED

An AI agent only for isolation, independence or on-demand use

Every extra agent adds cost, hand-over points and ways to fail. Everything else is ordinary code, which is cheaper, faster and can be tested exhaustively.

In practice Job Scout has three AI agents (a local reader, a writer, an independent reviewer) around a core of code that does all the reading, filtering and checking.

4 · TEXT FROM STRANGERS IS HOSTILE

Anything written by someone else is data, never instructions

Adverts, emails and web pages can carry hidden instructions aimed at AI (prompt injection). They are cleaned, held as data, and read by something with nothing to steal and nothing it can do.

In practice A model on the local PC with no tools and no internet reads each advert; code then keeps only facts that appear in the advert's own words.

5 · AUTHORITY IS EARNED

Read-only first; write access is earned with evidence

A system that can only read cannot break the things people rely on. Write access is added only where the friction is real, and only after it has been tested.

In practice The task app mirrors every source read-only, so a bug cannot complete forty tasks elsewhere. Job Scout may move a card on my board only after I have graded its reply reader; until then every move is a suggestion, logged, with Undo.

6 · NEVER LOSE, NEVER GUESS

Be lenient about reading, strict about writing

Where a decision belongs to a person, the system suggests and the person confirms. Where data looks damaged, the system stops writing and says so in plain words, rather than repairing it on a guess.

In practice The task app suggests likely duplicates but never merges them on its own. A half-synced task file produces a red banner naming the file and "nothing has been changed".

7 · EVIDENCE, NOT ASSERTION, IN PROPORTION

Nothing is reported as working until it has been seen to work

Tests are written before code. A separate agent that has not seen the work reviews the code before each delivery. AI components are measured against bars set in advance, and a red team plays the attacker every quarter. The effort is proportionate: a half-day item gets a short note, not a design document.

In practice Before one Job Scout delivery, the independent code review found eleven issues; nine were fixed before release and two were accepted, with reasons recorded. The local reader had to copy requirements and pay correctly for nine adverts in ten before it was used; it managed 21 of 22.

05 Two builds, briefly

Job Scout

An automated job search. Every morning it reads my job alerts and replies, reads the full adverts, scores each role against criteria I wrote, drafts a CV and letter for the best few, has a second agent check every claim against my career record, and puts the results on a private board where I decide what to do. It has run every day since 14 October 2026 and has never acted on my behalf. The companion case study covers it in full.

Task oversight app

One place to see and prioritise tasks that otherwise sit across Claude, Gemini, a task manager and notes on my own computer, with an Eisenhower matrix as the main view. It runs locally for a single user. Two guardrail decisions stand out. Tasks from AI conversations arrive by **deliberate capture, not scraping**: mining whole conversation histories for commitments would mean handling far more personal data than needed, and its guesses would erode trust in the list. And a QA pass found that **any website open in the browser could change tasks** through the local server; three checks now run before any request is handled, and the fix was retested.

06 What only real use finds

Some of the most useful lessons came from running Job Scout every day rather than from the design. The local PC's power-saving mode blocked the scheduled wake-up and switched off the network, so morning runs started late. A graphics setting left the local model running on the main processor at about a tenth of its normal speed, which only the reader's own speed warning revealed. Both are fixed and written into the setup notes.

The conclusion I draw

AI systems often fail where they meet the real world, not in their reasoning. They need the same operational care as any other service: monitoring, runbooks, and a close that is judged on a live run.

07 The paper trail

Every project leaves behind the documents a reviewer, an auditor or a new team member would ask for, kept with the code and updated at every sprint close.

DOCUMENT	WHAT IT ANSWERS
Decision log	Why it is built this way: each decision numbered and dated, with the reasoning and the cost, so the reasoning survives the conversation it came from.
System card	What each AI component is for and not for, what it sees, what it can do, how well it performs and where it fails.
Threat model	What needs protecting, who or what could cause harm, where text from strangers enters, and which control stops each threat.
Risk register	Each risk with its control, owner and last-checked date, organised by the four functions of the NIST AI Risk Management Framework: Govern, Map, Measure and Manage.
Agent catalogue	Every place an AI model makes a judgement: its model, instructions, permissions, checks and evaluation.
Runbook and delivery history	How to operate it when something goes wrong, and what each sprint planned, delivered and learned.

08 What this brings to a team

None of this is exotic. It is ordinary good practice; AI simply makes it much faster to follow, and just as much faster to skip. The value I would bring to a team or an organisation adopting AI is less any particular tool than the habit: writing down what must never happen before anything is built, deciding where data may go, giving AI only the authority it has earned, and proving claims rather than asserting them. That habit can be taught, and it outlasts any single build.

Rory McClelland · product leader, AI products and governance
hello@mcclelland.info · mcclelland.info

Companion case study:
Job Scout: a guarded, private job search